

Modular DBMS Engine -- Project Report

CMPE 321 -- Spring 2026 -- Project 3 -- Group 7

Beratcan Dogan (2021400132), Faik Ihsan Sudupak (2022400084)

Submission Links (spec section 16)

Explanation video (Layer-by-layer walkthrough, spec section 11):

`TODO_VIDEO_LINK_EXPLANATION`

Analysis video (Comparative experiments, spec section 12):

`TODO_VIDEO_LINK_ANALYSIS`

Source code (private GitHub repository, spec section 16):

<https://github.com/Ihsan2004/CmpE321-Project3>

1. Overview

We built a modular DBMS engine composed of four independent layers: Disk Space Manager, Buffer Manager, File and Index Manager, and Query Processor. Each layer lives in its own Python package and exports one class through `__init__.py`.

Layers communicate through Result objects defined in `results.py`: `PageResult`, `WriteResult`, `AllocResult`, `BufferResult`, `RecordResult`, and `OpResult`. `OpResult` also carries an optional value field for metadata operations such as file existence and page-count queries, so cross-layer calls can still return Result objects.

The engine is reconfigured by editing `config.json`. Replacement policy can be LRU or MRU, and index strategy can be `heap_scan`, `hash_index`, or `bplus_tree`. Output and data files are written next to `archive.py`, as required by spec section 13.

2. Layer 1 -- Disk Space Manager

The Disk Space Manager is the only component that performs real file I/O. It stores every relation or index as fixed-size pages and uses `seek()` for single-page reads and writes; it never loads an entire file into memory.

Every DSM-managed file has a DSM-owned physical header page at page 0. Upper layers see logical content pages starting at 0, and the DSM translates logical page N to physical page $N+1$. The header stores a magic value, the number of content pages, and a free list of deallocated logical pages.

We chose a free list over a global bitmap because whole-page deallocation is rare in our workload. Every physical read increments `read_count`. Every physical write increments `write_count` and invokes the `log_write` stub required by spec section 4.1, including header writes and allocation writes.

3. Layer 2 -- Buffer Manager

The Buffer Manager is an in-memory page cache between Layer 3 and the DSM. Its pool is an `OrderedDict` keyed by `(file_id, page_id)`. Lookup and move-to-end are constant time; victim selection scans until it finds an unpinned frame.

The right end of the `OrderedDict` is most recently used and the left end is least recently used. LRU

evicts from the left, while MRU evicts from the right. The active policy comes from `replacement_policy` in `config.json`.

Every `get_page` pins the frame and returns a `BufferResult`. On hits, the wrapped `PageResult` has `io_performed=False`. Callers must unpin the page, passing `dirty=True` after modifications. Dirty frames are written back on eviction and when `archive.py` calls `buffer.flush()` at the end of a run.

Layer 3 obtains file-existence, create-file, delete-file, allocate-page, deallocate-page, and page-count services through `BufferManager` wrappers. Those wrappers return `Result` objects and keep the DSM behind Layer 2.

4. Layer 3 -- File and Index Manager

Layer 3 understands schemas, records, slotted data pages, and indexes. It uses the Buffer Manager for page access and file metadata operations.

A Schema records the type name, ordered fields, and primary-key position. Integers are 4-byte signed little-endian values. Strings are 32-byte ASCII values padded with null bytes. For the sample house type, the record size is 108 bytes. With a 16-byte page header, 37 would fit geometrically, but `max_records_per_page` caps the page at 10 records.

Data pages use an unpacked slotted format. The 16-byte header stores `page_id`, `record_count`, and a uint16 slot bitmap. Slot `i` is stored at offset `16 + i * record_size`; deletes clear the bitmap bit and zero the slot.

The system catalog is `catalog.dat`, a paged file accessed through the Buffer Manager. Page 0 stores the number of types followed by serialized schemas. On restart, `Catalog.bootstrap()` reloads page 0 and rebuilds the in-memory type dictionary.

`heap_scan` is the baseline with no index. `hash_index` stores a 64-bucket directory in `<type>.hidx` page 0; directory entries point to bucket pages with overflow chaining. It supports equality lookups and falls back to heap scan for range queries. `bplus_tree` stores its header in `<type>.bidx` page 0, has internal and leaf node pages, and supports equality and primary-key range lookups through a leaf chain.

When an indexed strategy starts, the active index file is deleted if present and rebuilt from the data file. This prevents stale index results after a previous run mutated data under another strategy.

5. Layer 4 -- Query Processor

The Query Processor parses input commands, dispatches to Layer 3, writes query results and explain output to `output.txt`, writes stats snapshots to `stats_output.txt`, and appends every operation to `log.csv`.

Supported commands are create type, create record, delete record, search record, range_search, explain, stats, and stats reset. Bad commands and required failure conditions return failure Results where appropriate, are logged as failure, and do not crash the engine.

explain snapshots counters, writes a PLAN block, executes the wrapped DML command, writes a RESULT block, and then prints actual counter deltas. For range_search, the plan reports heap-scan fallback when `hash_index` is active or when `bplus_tree` cannot use the queried field.

stats_output.txt contains the five required categories: Disk I/O, Buffer Pool, Evictions, Index, and Records. log.csv uses int(time.time()),operation,status and is append-only across runs.

6. Experiments

All experiments use deterministic workloads generated by workload_generator.py with seed 42. The exact commands are documented in record.txt and automated by run_experiments.py.

Experiment 1 -- LRU vs MRU. Setup: 200 records, 50 queries, 8-frame buffer, heap_scan. Results: sequential: LRU 5105 I/Os / 17.4% hit rate, MRU 3245 I/Os / 51.4% hit rate. random: LRU 4697 I/Os / 19.8% hit rate, MRU 3023 I/Os / 52.5% hit rate. This demonstrates sequential flooding: LRU evicts pages just before the next scan needs them, while MRU preserves older pages.

Experiment 2 -- Index strategy comparison. Setup: 500 records, 100 queries, 16-frame LRU buffer. Equality: heap_scan 27378 I/Os, hash_index 14847, bplus_tree 15979. Range: heap_scan 29343, hash_index 19898 fallback, bplus_tree 17009. Hash wins equality; B+ tree wins range because it follows the leaf chain instead of scanning every page.

Experiment 3 -- Buffer pool sensitivity. Setup: 500 records, 200 random equality queries, bplus_tree, LRU. Buffer sizes 4, 8, 16, 32, 64 produce I/Os 19139, 17387, 16202, 11884, 1567 and hit rates 22.9%, 30.5%, 35.2%, 53.4%, 98.2%. The working set mostly fits around 64 frames, causing the large I/O drop.

7. Closing Notes

The implementation uses only the Python standard library. Type names, field names, and string values are ASCII-compatible; identifiers allow underscores to accept the project sample fields military_strength and spice_production, while string field values stay strictly alphanumeric.

The main validation path is reproducible from the submitted files: python3 archive.py config.json input.txt for functional runs and python3 run_experiments.py for the three experiment tables. Data files, index files, catalog.dat, and log.csv persist across runs.

Files produced by the engine next to archive.py: output.txt, log.csv, stats_output.txt, <type>.dat, <type>.hidx for hash_index, <type>.bidx for bplus_tree, and catalog.dat.